Machine Learning and Pattern Recognition

Neural Networks

Dr. Kewei "Isaac" Li

Department of Digital Engineering
Long Island University

March 29, 2025

Outline

- Final Project
- Introduction
- Basic Concept of Neuron
- Neural Networks

Final Project

- For final project, you may form a team of one, two or three students
- Final project proposal due on April 17.
  - Describe the motivation of your project and topic (problem)
  - Identify the data source and literature
  - Flowchart to illustrate the method/algorithm to solve the problem
- Final project report due on May 8.

Outline

- Final Project
- Introduction
- Basic Concept of Neuron
- Neural Networks

Introduction

- The concept of neural networks was founded on the 1943 model of neurons by McCulloch and Pitts and the learning ideas of Hebb
- There was a great deal of excitement, but not a lot of practical success.
- There were good training methods for linear functions, but no good way to train non-linear functions from data.
- Interest died out for a while, but was re-appeared in the 1980s when "back-propagation" was introduced to train the neural networks

Introduction

- By the mid-90s, the enthusiasm faded again because the training tended to be slow and was faced with issues of getting stuck in local optima
- With later improvements, they have become much more reliable and capable, and are now the method of choice in many applications
- Nowadays, there are many variations of neural networks, we will begin with the core "feed-forward" networks and "back-propagation" training

Introduction

We can view neural networks from several different perspectives:

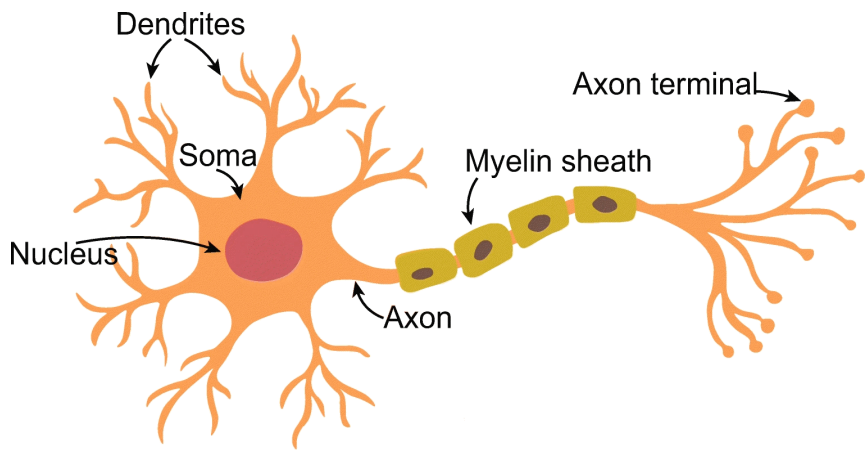
- An application of stochastic gradient descent for classification and regression with a potentially very rich hypothesis class.
- A brain-inspired network of neuron-like computing elements that learn distributed representations.
- A method for building applications that make predictions based on huge amounts of data in very complex domains

Outline

- Final Project
- Introduction
- Basic Concept of Neuron
- Neural Networks

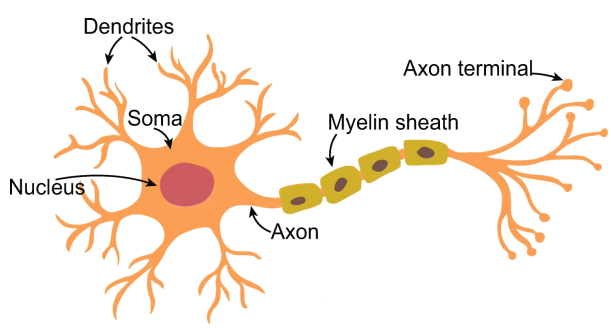
Basic Concept of Neuron

- The basic element of a neural network is a “neuron”. We also refer to a neuron as a “unit” or “node”.

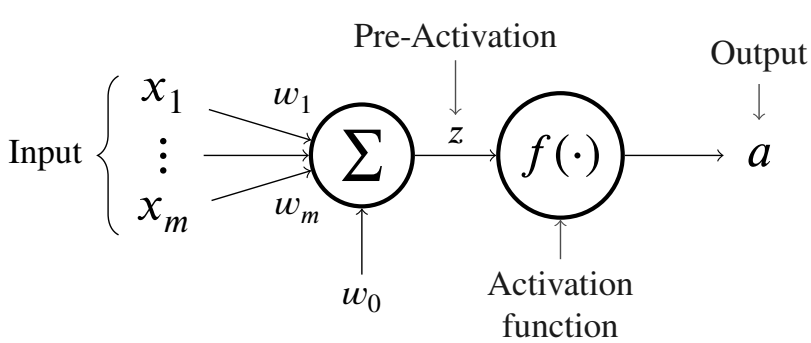


K.U. Mohanan, et. al., Applied Intelligence, 53, 2023

Basic Concept of Neuron



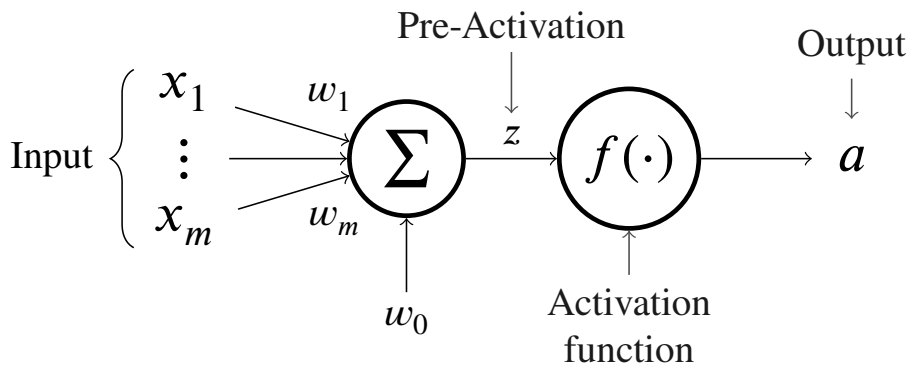
A model of a “neuron”.



K.U. Mohanan, et. al., Applied Intelligence, 53, 2023

Basic Concept of Neuron

- A model of a "neuron".



- This model is a non-linear function which maps an input $\mathbf{x} \in \mathbb{R}^m$ to an output $a \in \mathbb{R}$ with weights $\mathbf{w} \in \mathbb{R}^m$ and an offset w_0
- The activation function can be $f(z) = z$ or any other differentiable function.

Basic Concept of Neuron

- The mathematical model of the neuron is

$$a = f(z) = f\left(\mathbf{w}^{\mathrm{T}}\mathbf{x} + w_0\right)$$

- Before working with a whole network, we can consider how to train a single neuron.
- Consider a loss function $\mathcal{L}$(guess, actual) and a dataset $\{\mathbf{x}^{(i)}, y^{(i)}\}$,

$$J(\mathbf{w}, w_0) = \sum_{i=1}^{N} \mathcal{L}\left(f(\mathbf{x}^{(i)}; \mathbf{w}, w_0), y^{(i)}\right)$$

- We can minimize this objective function to find $\mathbf{w}$ and w_0 with the gradient descent method

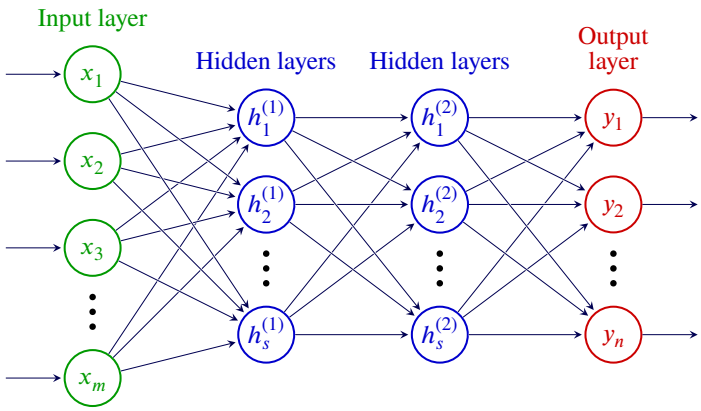
Basic Concept of Neuron

- We have already studied two special cases of the neuron:
  - Linear regressors with quadratic loss function
  - Linear logistic classifiers (LLCs) with NLL loss function
- The activation function for the LLC is $f(x) = \sigma(x)$, and for linear regression it is simply $f(x) = x$.

Outline

- Final Project
- Introduction
- Basic Concept of Neuron
- Neural Networks

Neural Networks

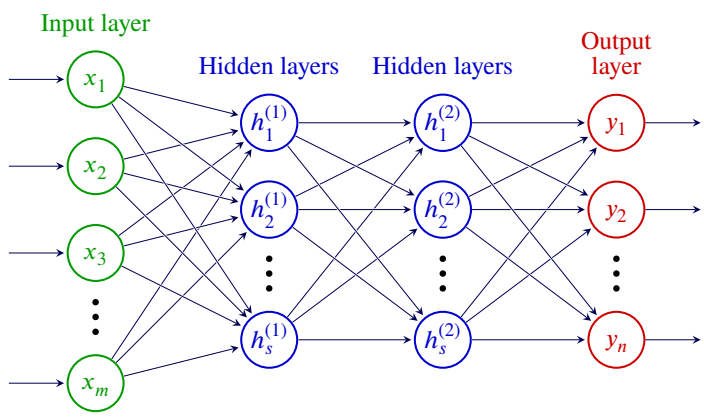


- A neural network is generally constructed out of multiple neurons. It takes in an input $\mathbf{x} \in \mathbb{R}^m$ and generates an output $\mathbf{a} \in \mathbb{R}^n$
- The inputs of each neuron might be elements of $\mathbf{x}$ and/or outputs of other neurons.
- The outputs are generated by n output units.

Neural Networks

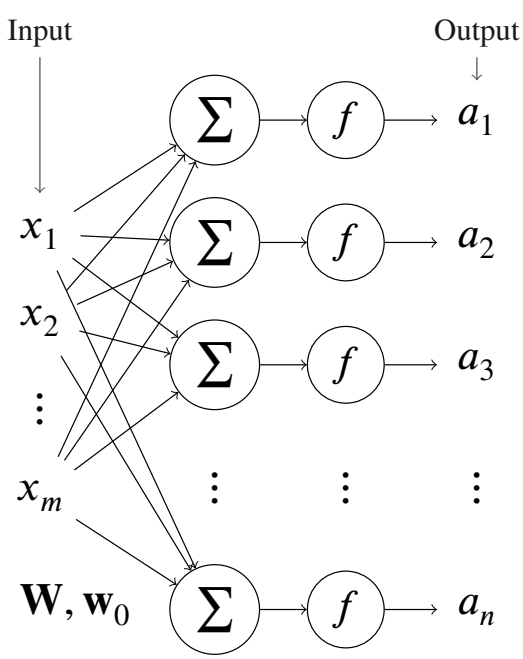
- We first consider the feed-forward networks in which the network is defined as follows:
  - The input to a neuron can never depend on its output
  - Data flows one way, from the inputs to the outputs
  - The function computed by the network is just a composition of the functions computed by the individual neurons

Neural Networks



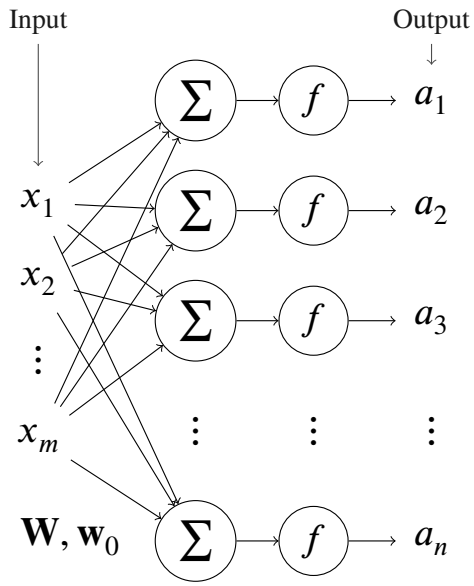
- In feed-forward neural networks, we usually organize them into layers
- A layer is a group of neurons that are essentially "in parallel": their inputs are outputs of neurons in the previous layer
- Their outputs are the input to the neurons in the next layer

Neural Networks: Single Layer



- A layer is a set of nodes that are not connected to each other
- A layer is fully connected if all of the inputs are connected to every node in the layer
- A layer has input $\mathbf{x} \in \mathbb{R}^m$ and output (also known as activation) $\mathbf{a} \in \mathbb{R}^n$
- For each node: we define a vector of weights ($\mathbf{w}$) and a single offset (w_0)
- For a layer, we have a matrix of weights, $\mathbf{W}$, and a vector of offsets $\mathbf{w}_0$.

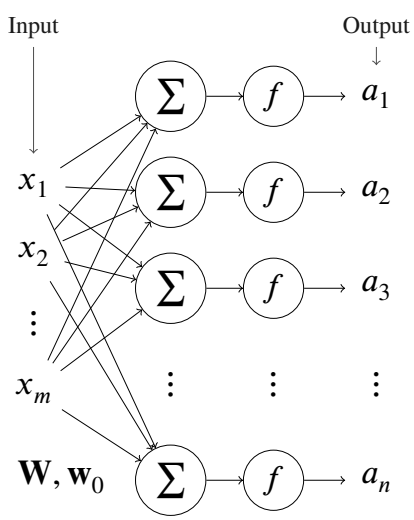
Neural Networks: Single Layer



For a layer of m inputs, n nodes, and n outputs

- $\mathbf{x}$: an $m \times 1$ column vector (input)
- $\mathbf{W}$: an $m \times n$ matrix ($\mathbf{W}^{\mathrm{T}}$ is $n \times m$)
- $\mathbf{w}_0$: an $n \times 1$ column vector
- $\mathbf{z} = \mathbf{W}^{\mathrm{T}}\mathbf{x} + \mathbf{w}_0$: an $n \times 1$ column vector (the pre-activation)
- $\mathbf{a}$: an $n \times 1$ column vector (the activation)

Neural Networks: Single Layer



- The output vector is

$$\mathbf{a} = f(\mathbf{z}) = f\left(\mathbf{W}^{\mathrm{T}}\mathbf{x} + \mathbf{w}_0\right)$$

- The activation function f is applied element-wise to the pre-activation ($\mathbf{z}$)

$$a_1 = f(z_1),\ a_2 = f(z_2),\ \cdots\ a_n = f(z_n)$$

Neural Networks: Many Layers

$$\xrightarrow{\mathbf{x}=\mathbf{a}^0} \underbrace{\boxed{\begin{matrix}\mathbf{W}^1\\ \mathbf{w}_0^1\end{matrix}} \xrightarrow{\mathbf{z}^1} \boxed{f^1}}_{Layer\ 1} \xrightarrow{\mathbf{a}^1} \cdots \xrightarrow{\mathbf{a}^{l-1}} \underbrace{\boxed{\begin{matrix}\mathbf{W}^l\\ \mathbf{w}_0^l\end{matrix}} \xrightarrow{\mathbf{z}^l} \boxed{f^l}}_{Layer\ l} \xrightarrow{\mathbf{a}^l} \cdots \xrightarrow{\mathbf{a}^{L-1}} \underbrace{\boxed{\begin{matrix}\mathbf{W}^L\\ \mathbf{w}_0^L\end{matrix}} \xrightarrow{\mathbf{z}^L} \boxed{f^L}}_{Layer\ L} \xrightarrow{\mathbf{a}^L}$$

- A neural network generally combines multiple layers, most typically by feeding the outputs of one layer into the inputs of another layer
- We use l to denote a layer, and let m^l be the number of inputs to the l-th layer and n^l be the number of outputs from the layer
- Then, $\mathbf{W}^l$ and $\mathbf{w}_0^l$ are of shape $m^l \times n^l$ and $n^l \times 1$, respectively

Neural Networks: Many Layers

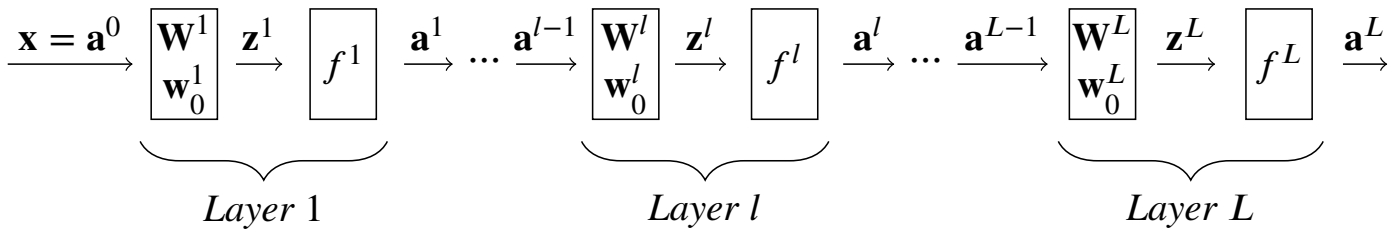
$$\xrightarrow{\mathbf{x}=\mathbf{a}^0} \underbrace{\boxed{\begin{matrix}\mathbf{W}^1\\ \mathbf{w}_0^1\end{matrix}} \xrightarrow{\mathbf{z}^1} \boxed{f^1}}_{Layer\ 1} \xrightarrow{\mathbf{a}^1} \cdots \xrightarrow{\mathbf{a}^{l-1}} \underbrace{\boxed{\begin{matrix}\mathbf{W}^l\\ \mathbf{w}_0^l\end{matrix}} \xrightarrow{\mathbf{z}^l} \boxed{f^l}}_{Layer\ l} \xrightarrow{\mathbf{a}^l} \cdots \xrightarrow{\mathbf{a}^{L-1}} \underbrace{\boxed{\begin{matrix}\mathbf{W}^L\\ \mathbf{w}_0^L\end{matrix}} \xrightarrow{\mathbf{z}^L} \boxed{f^L}}_{Layer\ L} \xrightarrow{\mathbf{a}^L}$$

- The input to layer l is the output from layer $l-1$, so we have $m^l = n^{l-1}$
- As a result, $\mathbf{a}^{l-1}$ is of shape $n^{l-1} \times 1$, or equivalently $m^l \times 1$
- Let f^l be the activation function of layer l. Then,

$$\mathbf{a}^l = f^l(\mathbf{z}^l) \quad \text{and} \quad \mathbf{z}^l = (\mathbf{W}^l)^{\mathrm{T}}\mathbf{x} + \mathbf{w}_0^l$$

- The pre-activation ($\mathbf{z}^l$) and activation ($\mathbf{a}^l$) are both $n^l \times 1$ vector

Neural Networks: Multiple Layers



- In this multi-layer network, there are two blocks in each layer
- One representing the linear part of the operation
- The other representing the non-linear activation function.

Neural Networks: Activation Function

- At first, we consider the case when $f(\mathbf{z}) = \mathbf{z}$. Then, in a network with L layers (we omitted $\mathbf{w}_0$ for simplicity),

$$\mathbf{a}^L = (\mathbf{W}^L)^{\mathrm{T}}\mathbf{a}^{L-1} = (\mathbf{W}^L)^{\mathrm{T}}(\mathbf{W}^{L-1})^{\mathrm{T}} \cdots (\mathbf{W}^1)^{\mathrm{T}}\mathbf{x}$$

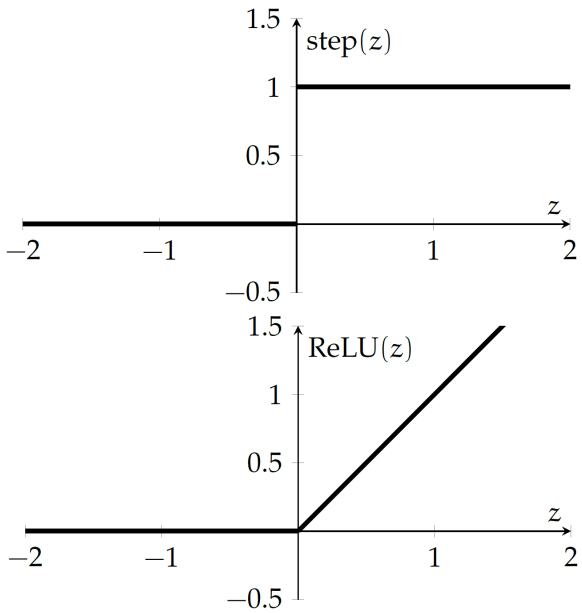
- Multiplying all the weight matrices, we find that

$$\mathbf{a}^L = \mathbf{W}^{\mathrm{Total}}\mathbf{x}$$

  which is a linear function of $\mathbf{x}$
- Having all those layers did not change the representational capacity of the network: the non-linearity of the activation function is crucial

Neural Networks: Activation Function

Now, we introduce a few common non-linear activation functions.



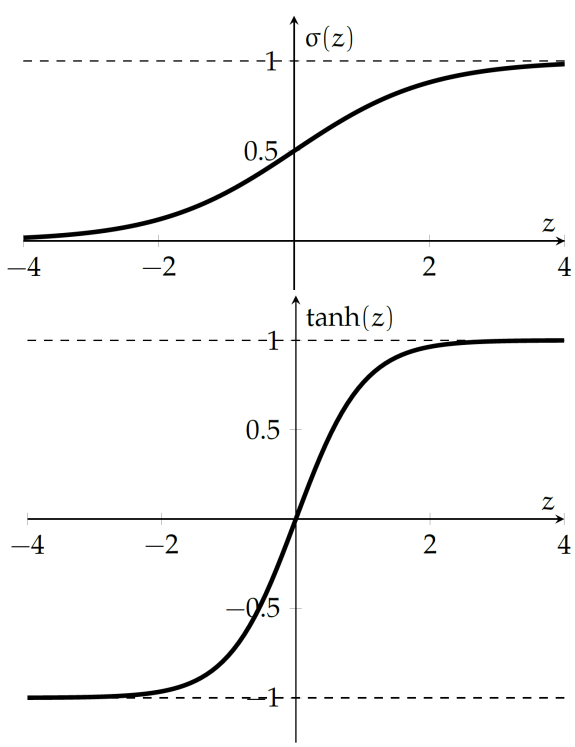
- **Step function**

$$\text{step}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{otherwise} \end{cases}$$

- **Rectified linear unit (ReLU)**:

$$\text{ReLU}(z) = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{otherwise} \end{cases}$$

Neural Networks: Activation Function



- **Sigmoid or logistic function**: This can sometimes be interpreted as probability, because for any value of z the output is in $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Hyperbolic tangent**: Always in the range $(-1, 1)$:

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

Neural Networks: Activation Function

- **Softmax function**: Takes a whole vector $\mathbf{z} \in \mathbb{R}^n$ and generates as output a vector $\mathbf{a} \in \mathbb{R}^n$

$$\mathbf{a} = \mathrm{softmax}(\mathbf{z}) = \begin{bmatrix} \frac{\exp(z_1)}{s} \\ \frac{\exp(z_2)}{s} \\ \vdots \\ \frac{\exp(z_n)}{s} \end{bmatrix}, \quad \text{where } s = \sum_{i=1}^{n} \exp(z_i)$$

Neural Networks: Activation Function

- Originally, the step function was introduced as an activation.
- However, it does not work well with gradient-descent method in searching for a good set of the weights
- They have been replaced by the $\mathrm{softmax}$, ReLU, and $\tanh$ activation functions.
- ReLU activations are especially common in internal ("hidden") layers

Neural Networks: Activation Function

- $\mathrm{Sigmoid}$ activations are common for the output for binary classification
- $\mathrm{softmax}$ activations are common for the output for multi-class classification
- Different loss functions have different assumptions about the range of their input values
- Different activation functions will produce output in different ranges

Neural Networks: Loss Functions and Activation Function

- When designing a neural network, it's important to make these ranges match with each other.
- In particular, we consider matching loss functions with the activation function in the last layer f^L.
- Here is a table of loss function and matching activation:

Loss	f^L	Task
Squared	Linear	Regression
NLL	Sigmoid	Binary Classification
NLLM	Softmax	Multi-class Classification

Neural Networks: Error Back-Propagation

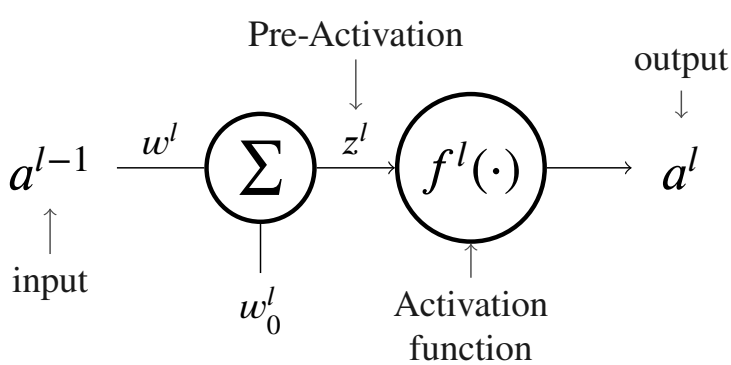
- We will train neural networks using gradient descent methods.
  - **Batch gradient descent**, we sum up the gradient over all the points
  - **Stochastic gradient descent (SGD)**, we take a small step with respect to the gradient considering a single point at a time
- Next, we compute the gradient of the loss function with respect to the weights
- For any data point $(\mathbf{x}, \mathbf{y})$, we need to compute

$$\nabla_{\mathbf{W}} \mathcal{L}(\mathrm{NN}(\mathbf{x}; \mathbf{W}), \mathbf{y})$$

where $\mathbf{W}$ represents all weights $(\mathbf{W}^l, \mathbf{w}_0^l)$ in all the layers

Neural Networks: Error Back-Propagation

- First, suppose everything in our neural network is one-dimensional.



- So, there are $m^l = 1$ input and $n^l = 1$ output at every layer. Then, for layer l, we have:

$$a^l = f^l(z^l), \quad z^l = w^l a^{l-1} + w_0^l$$

Neural Networks: Error Back-Propagation

- Similarly, for the layer $(l-1)$,

$$a^{l-1} = f^{l-1}(z^{l-1}), \quad z^{l-1} = w^{l-1}a^{l-2} + w_0^{l-1}$$

- And the last layer L,

$$a^L = f^L(z^L), \quad z^L = w^L a^{L-1} + w_0^L$$

- To compute the derivatives of the loss function with respect to the weights. First, we consider the last layer when $l = L$,

$$\frac{\partial}{\partial w^L}\mathcal{L} = \frac{\partial \mathcal{L}}{\partial a^L} \cdot \frac{\partial a^L}{\partial z^L} \cdot \frac{\partial z^L}{\partial w^L} = \frac{\partial \mathcal{L}}{\partial a^L} \cdot (f^L)'(z^L) \cdot a^{L-1}$$

Neural Networks: Error Back-Propagation

- Now, for general case l:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w^l} &= \frac{\partial \mathcal{L}}{\partial a^L} \cdot \frac{\partial a^L}{\partial z^L} \cdot \frac{\partial z^L}{\partial a^{L-1}} \cdot \frac{\partial a^{L-1}}{\partial z^{L-1}} \cdots \frac{\partial z^{l+1}}{\partial a^l} \cdot \frac{\partial a^l}{\partial z^l} \cdot \frac{\partial z^l}{\partial w^l} \\ &= \frac{\partial \mathcal{L}}{\partial a^L} \cdot (f^L)'(z^L) \cdot w^L \cdot (f^{L-1})'(z^{L-1}) \cdots w^{l+1} \cdot (f^l)'(z^l) \cdot a^{l-1} \\ &= \frac{\partial \mathcal{L}}{\partial z^l} \cdot a^{l-1}\end{aligned}$$

- Although we solved for all the other terms in the product, we haven't solved for $\partial \mathcal{L}/\partial a^L$ because it depends on the loss function
- Once we choose a loss function, we will be able to compute this derivative.

Neural Networks: Error Back-Propagation

- We can rewrite our result in the reverse order:

$$\frac{\partial \mathcal{L}}{\partial w^l} = a^{l-1} \cdot \frac{\partial \mathcal{L}}{\partial z^l}$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial z^l} &= \frac{\partial a^l}{\partial z^l} \cdot \frac{\partial z^{l+1}}{\partial a^l} \cdots \frac{\partial a^{L-1}}{\partial z^{L-1}} \cdot \frac{\partial z^L}{\partial a^{L-1}} \cdot \frac{\partial a^L}{\partial z^L} \cdot \frac{\partial \mathcal{L}}{\partial a^L} \\ &= \frac{\partial a^l}{\partial z^l} \cdot w^{l+1} \cdots \frac{\partial a^{L-1}}{\partial z^{L-1}} \cdot w^L \cdot \frac{\partial a^L}{\partial z^L} \cdot \frac{\partial \mathcal{L}}{\partial a^L}\end{aligned}$$

Neural Networks: Error Back-Propagation

$$\xrightarrow{\mathbf{x} = \mathbf{a}^0} \underbrace{\boxed{\begin{matrix}\mathbf{W}^1 \\ \mathbf{w}_0^1\end{matrix}} \xrightarrow{\mathbf{z}^1} \boxed{f^1}}_{Layer\ 1} \xrightarrow{\mathbf{a}^1} \cdots \xrightarrow{\mathbf{a}^{l-1}} \underbrace{\boxed{\begin{matrix}\mathbf{W}^l \\ \mathbf{w}_0^l\end{matrix}} \xrightarrow{\mathbf{z}^l} \boxed{f^l}}_{Layer\ l} \xrightarrow{\mathbf{a}^l} \cdots \xrightarrow{\mathbf{a}^{L-1}} \underbrace{\boxed{\begin{matrix}\mathbf{W}^L \\ \mathbf{w}_0^L\end{matrix}} \xrightarrow{\mathbf{z}^L} \boxed{f^L}}_{Layer\ L} \xrightarrow{\mathbf{a}^L}$$

- Now for the general case, we have the loss function (scalar)

$$\mathcal{L} = \mathcal{L}(\mathrm{NN}(\mathbf{x}, \underbrace{\{\mathbf{W}^l, \mathbf{w}_0^l | l = 1, 2, \cdots, L\}}_{\text{Set of all coefficients}}), \mathbf{y})$$

- And, the output vector for layer l,

$$\mathbf{a}^l = f^l(\mathbf{z}^l) \quad \text{and} \quad \mathbf{z}^l = (\mathbf{W}^l)^{\mathrm{T}}\mathbf{x} + \mathbf{w}_0^l$$

Neural Networks: Error Back-Propagation

$$\xrightarrow{\mathbf{x}=\mathbf{a}^0} \underbrace{\boxed{\begin{matrix}\mathbf{W}^1\\ \mathbf{w}_0^1\end{matrix}} \xrightarrow{\mathbf{z}^1} \boxed{f^1}}_{Layer\ 1} \xrightarrow{\mathbf{a}^1} \cdots \xrightarrow{\mathbf{a}^{l-1}} \underbrace{\boxed{\begin{matrix}\mathbf{W}^l\\ \mathbf{w}_0^l\end{matrix}} \xrightarrow{\mathbf{z}^l} \boxed{f^l}}_{Layer\ l} \xrightarrow{\mathbf{a}^l} \cdots \xrightarrow{\mathbf{a}^{L-1}} \underbrace{\boxed{\begin{matrix}\mathbf{W}^L\\ \mathbf{w}_0^L\end{matrix}} \xrightarrow{\mathbf{z}^L} \boxed{f^L}}_{Layer\ L} \xrightarrow{\mathbf{a}^L}$$

- Now, for layer l, the derivatives of the loss function with respect to the weights matrix $\mathbf{W}^l$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} = \mathbf{a}^{l-1}\left(\frac{\partial \mathcal{L}}{\partial \mathbf{z}^l}\right)^{\mathrm{T}}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^l} = \frac{\partial \mathbf{a}^l}{\partial \mathbf{z}^l} \cdot \frac{\partial \mathbf{z}^{l+1}}{\partial \mathbf{a}^l} \cdots \frac{\partial \mathbf{a}^{L-1}}{\partial \mathbf{z}^{L-1}} \cdot \frac{\partial \mathbf{z}^L}{\partial \mathbf{a}^{L-1}} \cdot \frac{\partial \mathbf{a}^L}{\partial \mathbf{z}^L} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{a}^L}$$

Neural Networks: Error Back-Propagation

- Recall that $\mathbf{z}^l$ is an $n^l \times 1$ column vector

$$\mathbf{z}^l = [z_1^l, z_2^l, \ldots, z_{n^l}^l]^{\mathrm{T}}$$

- Also, $\mathbf{W}^l$ is an $m^l \times n^l$ matrix

$$\mathbf{W}^l = \begin{bmatrix} W_{11}^l & W_{12}^l & \cdots & W_{1n^l}^l \\ W_{21}^l & W_{22}^l & \cdots & W_{2n^l}^l \\ \vdots & \vdots & \ddots & \vdots \\ W_{m^l1}^l & W_{m^l2}^l & \cdots & W_{m^ln^l}^l \end{bmatrix}$$

Neural Networks: Error Back-Propagation

$$\xrightarrow{\mathbf{x}=\mathbf{a}^0} \underbrace{\boxed{\begin{matrix}\mathbf{W}^1\\ \mathbf{w}_0^1\end{matrix}} \xrightarrow{\mathbf{z}^1} \boxed{f^1}}_{Layer\ 1} \xrightarrow{\mathbf{a}^1} \cdots \xrightarrow{\mathbf{a}^{l-1}} \underbrace{\boxed{\begin{matrix}\mathbf{W}^l\\ \mathbf{w}_0^l\end{matrix}} \xrightarrow{\mathbf{z}^l} \boxed{f^l}}_{Layer\ l} \xrightarrow{\mathbf{a}^l} \cdots \xrightarrow{\mathbf{a}^{L-1}} \underbrace{\boxed{\begin{matrix}\mathbf{W}^L\\ \mathbf{w}_0^L\end{matrix}} \xrightarrow{\mathbf{z}^L} \boxed{f^L}}_{Layer\ L} \xrightarrow{\mathbf{a}^L}$$

- Recall that $\mathbf{z}^l = (\mathbf{W}^l)^{\mathrm{T}}\mathbf{a}^{l-1} + \mathbf{w}_0^l$, then

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^l}\frac{\partial \mathbf{z}^l}{\partial \mathbf{W}^l}$$

- Now, because $\mathbf{z}^l$ has n^l elements, we write above equation as

$$\frac{\partial \mathcal{L}}{\partial W_{i,j}^l} = \frac{\partial \mathcal{L}}{\partial z_1^l}\frac{\partial z_1^l}{\partial W_{i,j}^l} + \frac{\partial \mathcal{L}}{\partial z_2^l}\frac{\partial z_2^l}{\partial W_{i,j}^l} + \cdots + \frac{\partial \mathcal{L}}{\partial z_{n^l}^l}\frac{\partial z_{n^l}^l}{\partial W_{i,j}^l}$$

Neural Networks: Error Back-Propagation

- Next, we write $\mathbf{z}^l$ in matrix form,

$$\mathbf{z}^l = (\mathbf{W}^l)^{\mathrm{T}}\mathbf{a}^{l-1} + \mathbf{w}_0^l = \begin{bmatrix} W_{11}^l & W_{21}^l & \cdots & W_{m^l 1}^l \\ W_{12}^l & W_{22}^l & \cdots & W_{m^l 2}^l \\ \vdots & \vdots & \ddots & \vdots \\ W_{1n^l}^l & W_{2n^l}^l & \cdots & W_{m^l n^l}^l \end{bmatrix} \begin{bmatrix} a_1^{l-1} \\ a_2^{l-1} \\ \vdots \\ a_{m^l}^{l-1} \end{bmatrix} + \begin{bmatrix} w_{0,1}^l \\ w_{0,2}^l \\ \vdots \\ w_{0,n^l}^l \end{bmatrix}$$

- From that, we know the first element of $\mathbf{z}^l$ is

$$z_1^l = W_{11}^l a_1^{l-1} + W_{21}^l a_2^{l-1} + \cdots + W_{m^l 1}^l a_{m^l}^{l-1} + w_{0,1}^l$$

- For a general case, the k-th element of $\mathbf{z}^l$ is

$$z_k^l = W_{1k}^l a_1^{l-1} + W_{2k}^l a_2^{l-1} + \cdots + W_{m^l k}^l a_{m^l}^{l-1} + w_{0,k}^l$$

Neural Networks: Error Back-Propagation

- From that, we have

$$\frac{\partial z_k^l}{\partial W_{ij}^l} = \begin{cases} a_i^{l-1}, & \text{if } k = j \\ 0, & \text{otherwise} \end{cases}$$

- Now, we eliminate all the zero terms and obtain,

$$\frac{\partial \mathcal{L}}{\partial W_{i,j}^l} = \frac{\partial \mathcal{L}}{\partial z_1^l}\frac{\partial z_1^l}{\partial W_{i,j}^l} + \frac{\partial \mathcal{L}}{\partial z_2^l}\frac{\partial z_2^l}{\partial W_{i,j}^l} + \cdots + \frac{\partial \mathcal{L}}{\partial z_{n^l}^l}\frac{\partial z_{n^l}^l}{\partial W_{i,j}^l} = \frac{\partial \mathcal{L}}{\partial z_j^l}\frac{\partial z_j^l}{\partial W_{i,j}^l} = \frac{\partial \mathcal{L}}{\partial z_j^l} a_i^{l-1}$$

- Also, we have

$$\frac{\partial z_k^l}{\partial w_{0,j}^l} = \begin{cases} 1, & \text{if } k = j \\ 0, & \text{otherwise} \end{cases}$$

Neural Networks: Error Back-Propagation

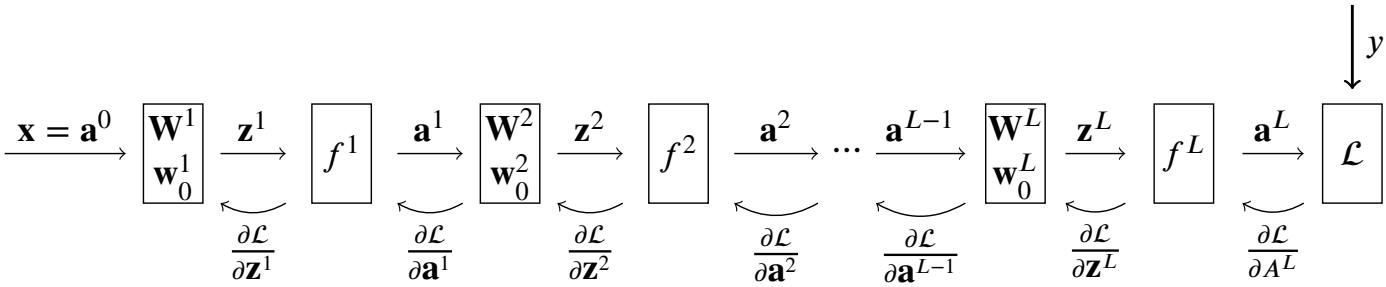
- We can also write it as matrices,

$$\underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{W}^l}}_{m^l \times n^l} = \begin{bmatrix} a_1^{l-1} \\ a_2^{l-1} \\ \vdots \\ a_{m^l}^{l-1} \end{bmatrix} \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial z_1^l} & \frac{\partial \mathcal{L}}{\partial z_2^l} & \cdots & \frac{\partial \mathcal{L}}{\partial z_{n^l}^l} \end{bmatrix} = \mathbf{a}^{l-1} \left(\frac{\partial \mathcal{L}}{\partial \mathbf{z}^l}\right)^{\mathrm{T}}$$

- Similarly, we have

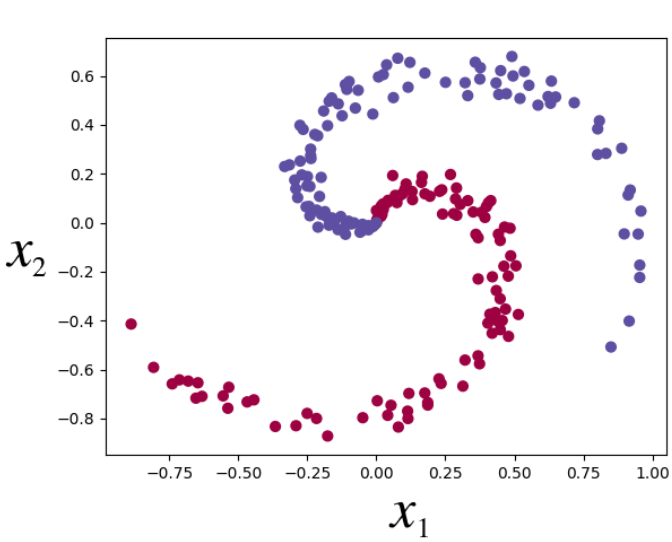
$$\underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{w}_0^l}}_{n^l \times 1} = \underbrace{\frac{\partial \mathbf{z}^l}{\partial \mathbf{w}_0^l}}_{n^l \times n^l} \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{z}^l}}_{n^l \times 1} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^l} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial z_1^l} & \frac{\partial \mathcal{L}}{\partial z_2^l} & \cdots & \frac{\partial \mathcal{L}}{\partial z_{n^l}^l} \end{bmatrix}^{\mathrm{T}}$$

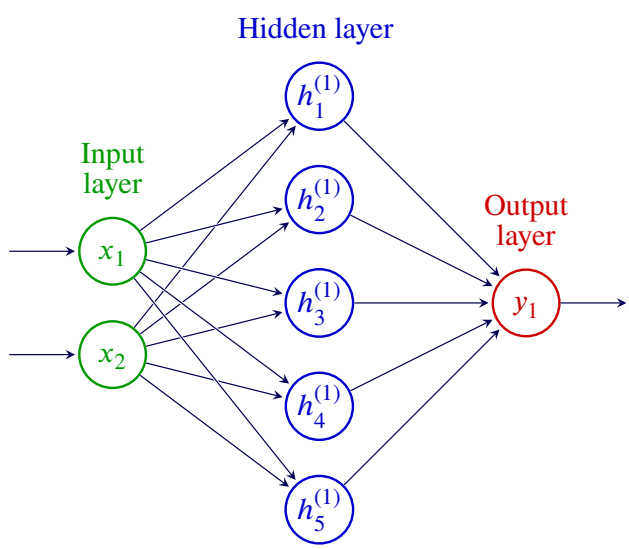
Neural Networks: Error Back-Propagation



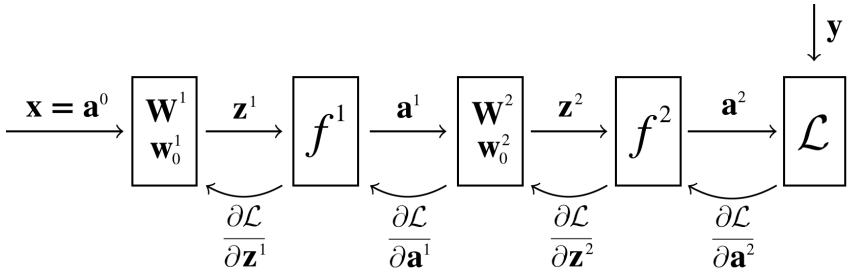
- To compute gradients of the loss, we first do a forward pass to compute all the $\mathbf{a}$ and $\mathbf{z}$ values at each layer, and the actual loss
- Then, we can work backward and compute the gradient of the loss in each layer, starting at layer L and going back to layer 1
- This general process of computing the gradients of the loss with respect to the weights is called error back-propagation.

Neural Networks: An Example with One Hidden Layer





Neural Networks: An Example with One Hidden Layer



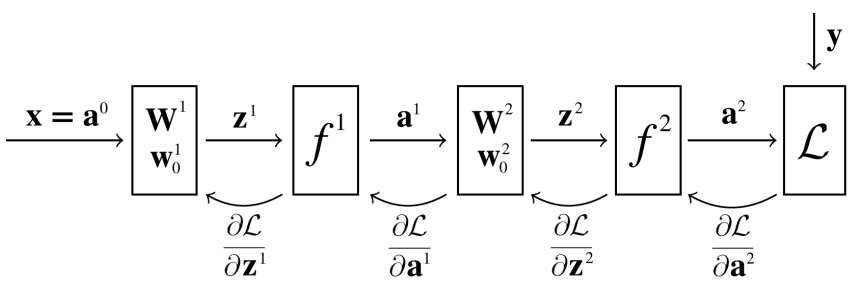
- Since our neural network has two layers, we choose the activation functions first.
- We define the activation function for the hidden layer as,

$$\mathbf{a}^1 = f^1(\mathbf{z}^1) = \mathrm{ReLU}(\mathbf{z}^1) = \max(0, \mathbf{z}^1)$$

- And then for the output layer:

$$\mathbf{a}^2 = f^2(\mathbf{z}^2) = \mathrm{softmax}(\mathbf{z}^2)$$

Neural Networks: An Example with One Hidden Layer



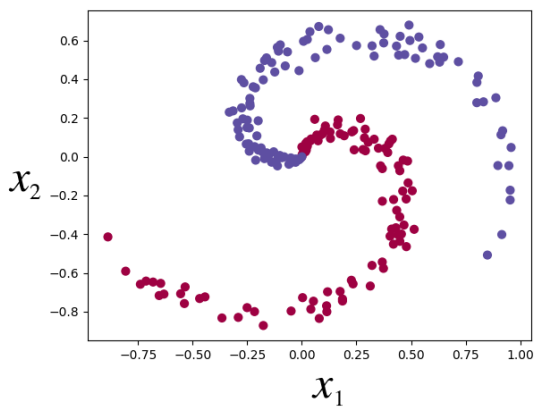
- Now, we compute $\mathbf{a}$ and $\mathbf{z}$ for the hidden layer (1)

$$\mathbf{z}^1 = (\mathbf{W}^1)^{\mathrm{T}}\mathbf{x} + \mathbf{w}_0^1, \quad \mathbf{a}^1 = \max(0, \mathbf{z}^1)$$

- Now, we compute $\mathbf{a}$ and $\mathbf{z}$ for the output layer (2)

$$\mathbf{z}^2 = (\mathbf{W}^2)^{\mathrm{T}}\mathbf{a}^1 + \mathbf{w}_0^2, \quad \mathbf{a}^2 = \mathrm{softmax}(\mathbf{z}^2)$$

Neural Networks: An Example with One Hidden Layer



- As an example, we assume following input and label data ($y \in \{0, 1\}$),

$$\mathbf{x} = \begin{bmatrix} 0.08 \\ 0.73 \end{bmatrix}, \quad y = 1$$

- And the initial guess for coefficients

$$\mathbf{W}^1 = \begin{bmatrix} 0.18 & 0.04 & 0.09 & 0.22 & 0.19 \\ -0.09 & 0.09 & -0.02 & -0.01 & 0.04 \end{bmatrix},$$
$$\mathbf{w}_0^1 = \begin{bmatrix} 0.01 & 0.15 & 0.07 & 0.01 & 0.04 \end{bmatrix}^{\mathrm{T}}$$
$$\mathbf{W}^2 = \begin{bmatrix} 0.03 & -0.02 & -0.08 & 0.07 & -0.07 \\ 0.15 & 0.03 & -0.26 & 0.09 & 0.23 \end{bmatrix}^{\mathrm{T}},$$
$$\mathbf{w}_0^2 = \begin{bmatrix} -0.15 & 0.004 \end{bmatrix}^{\mathrm{T}}$$

Neural Networks: An Example with One Hidden Layer

- Now, we compute $\mathbf{z}^1$ for the hidden layer (1)

$$\mathbf{z}^1 = (\mathbf{W}^1)^{\mathrm{T}}\mathbf{x} + \mathbf{w}_0^1 = \begin{bmatrix} 0.18 & -0.09 \\ 0.04 & 0.09 \\ 0.09 & -0.02 \\ 0.22 & -0.01 \\ 0.19 & 0.04 \end{bmatrix} \begin{bmatrix} 0.08 \\ 0.73 \end{bmatrix} + \begin{bmatrix} 0.01 \\ 0.15 \\ 0.07 \\ 0.01 \\ 0.04 \end{bmatrix} = \begin{bmatrix} -0.04 \\ 0.22 \\ 0.07 \\ 0.02 \\ 0.09 \end{bmatrix}$$

- Also the vector $\mathbf{a}^1$

$$\mathbf{a}^1 = \max(0, \mathbf{z}^1) = \begin{bmatrix} 0.0 & 0.22 & 0.07 & 0.02 & 0.09 \end{bmatrix}^{\mathrm{T}}$$

Neural Networks: An Example with One Hidden Layer

- Similarly, we obtain $\mathbf{z}^2$ for the output layer (2)

$$\mathbf{z}^2 = (\mathbf{W}^2)^{\mathrm{T}}\mathbf{a}^1 + \mathbf{w}_0^2$$

$$= \begin{bmatrix} 0.03 & -0.02 & -0.08 & 0.07 & -0.07 \\ 0.15 & 0.03 & -0.26 & 0.09 & 0.23 \end{bmatrix} \begin{bmatrix} 0.0 \\ 0.22 \\ 0.07 \\ 0.02 \\ 0.09 \end{bmatrix} + \begin{bmatrix} -0.15 \\ 0.004 \end{bmatrix} = \begin{bmatrix} -0.16 \\ 0.02 \end{bmatrix}$$

- Also the vector $\mathbf{a}^2$

$$\mathbf{a}^2 = \mathrm{softmax}(\mathbf{z}^2) = \begin{bmatrix} 0.46 \\ 0.54 \end{bmatrix}$$

which indicates that the estimated class of this input data ($\mathbf{x}$) is 1

Neural Networks: An Example with One Hidden Layer

- Now, we define the NLL loss for a single data point as,

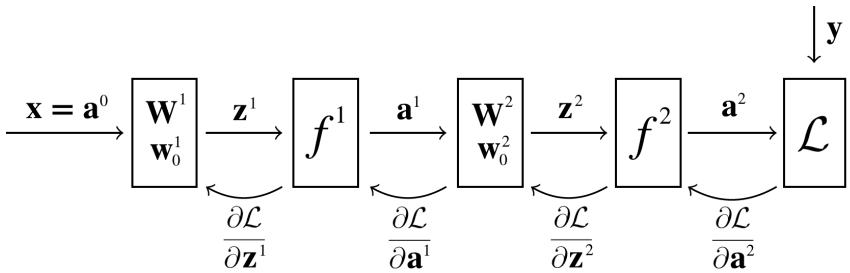
$$\mathcal{L}_{(1)} = -\ln a_c^L$$

where a_c^L is the element of $\mathbf{a}^L$ that represents the correct probability

- For example, if $\mathbf{a}^L = [0.46,\ 0.54]^{\mathrm{T}}$ and the label for this data point is $y = 1$, then $a_c^L = 0.54$. If $y = 0$, then $a_c^L = 0.46$.
- Thus, the NLL loss over a batch of data of size N becomes:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}_{(i)} = -\frac{1}{N}\sum_{i=1}^{N} \ln a_{c,i}^L$$

Neural Networks: An Example with One Hidden Layer



- Now, for layer 2, the derivatives of the loss function with respect to the weights matrix $\mathbf{W}^2$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^2} = \mathbf{a}^1 \left(\frac{\partial \mathcal{L}}{\partial \mathbf{z}^2}\right)^{\mathrm{T}}, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{z}^2} = \frac{\partial \mathbf{a}^2}{\partial \mathbf{z}^2} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{a}^2}$$

- Similarly, for layer 1,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^1} = \mathbf{x} \left(\frac{\partial \mathcal{L}}{\partial \mathbf{z}^1}\right)^{\mathrm{T}}, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{z}^1} = \frac{\partial \mathbf{a}^1}{\partial \mathbf{z}^1} \cdot \frac{\partial \mathbf{z}^2}{\partial \mathbf{a}^1} \cdot \frac{\partial \mathbf{a}^2}{\partial \mathbf{z}^2} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{a}^2}$$

Neural Networks: An Example with One Hidden Layer

- First, the derivative of the loss with respect to $\mathbf{a}^2$ for one data point:

$$\frac{\partial \mathcal{L}_{(1)}}{\partial \mathbf{a}^2} = -\frac{\partial}{\partial \mathbf{a}^2} \ln a_c^2 = \begin{bmatrix} -\frac{\partial \ln a_c^2}{\partial a_1^2} \\ -\frac{\partial \ln a_c^2}{\partial a_2^2} \end{bmatrix} = \begin{cases} \left[-\frac{1}{a_1^2}, 0\right]^{\mathrm{T}} & \text{when } y = 0 \\ \left[0, -\frac{1}{a_2^2}\right]^{\mathrm{T}} & \text{when } y = 1 \end{cases}$$

- Next, the derivative of $\mathbf{a}^2$ with respect to $\mathbf{z}^2$ for one data point:

$$\frac{\partial \mathbf{a}^2}{\partial \mathbf{z}^2} = \frac{\partial}{\partial \mathbf{z}^2} \operatorname{softmax}(\mathbf{z}^2) = \begin{bmatrix} \frac{\partial a_1^2}{\partial z_1^2} & \frac{\partial a_1^2}{\partial z_2^2} \\ \frac{\partial a_2^2}{\partial z_1^2} & \frac{\partial a_2^2}{\partial z_2^2} \end{bmatrix} = \begin{bmatrix} a_1^2(1 - a_1^2) & -a_1^2 a_2^2 \\ -a_2^2 a_1^2 & a_2^2(1 - a_2^2) \end{bmatrix}$$

Neural Networks: An Example with One Hidden Layer

- The derivative of the loss with respect to $\mathbf{z}^2$ for one data point:

$$\frac{\partial \mathcal{L}_{(1)}}{\partial \mathbf{z}^2} = \left(\frac{\partial \mathbf{a}^2}{\partial \mathbf{z}^2}\right)^{\mathrm{T}} \cdot \frac{\partial \mathcal{L}_{(1)}}{\partial \mathbf{a}^2} = \begin{bmatrix} \frac{\partial a_1^2}{\partial z_1^2} & \frac{\partial a_2^2}{\partial z_1^2} \\ \frac{\partial a_1^2}{\partial z_2^2} & \frac{\partial a_2^2}{\partial z_2^2} \end{bmatrix} \begin{bmatrix} \frac{\partial \mathcal{L}_{(1)}}{\partial a_1^2} \\ \frac{\partial \mathcal{L}_{(1)}}{\partial a_2^2} \end{bmatrix} = \begin{cases} \left[a_1^2 - 1, a_2^2\right]^{\mathrm{T}} & \text{when } y = 0 \\ \left[a_2^1, a_2^2 - 1\right]^{\mathrm{T}} & \text{when } y = 1 \end{cases}$$

- Note, the label data y is known for the training data set
- Then, the total loss of the data set,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^2} = \frac{1}{N} \sum_{i=1}^{N} \frac{\partial \mathcal{L}_{(i)}}{\partial \mathbf{z}_{(i)}^2} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial z_1^2} \\ \frac{\partial \mathcal{L}}{\partial z_2^2} \end{bmatrix}$$

Neural Networks: An Example with One Hidden Layer

- Next, the derivative of the total loss with respect to $\mathbf{W}^2$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^2} = \mathbf{a}^1 \left(\frac{\partial \mathcal{L}}{\partial \mathbf{z}^2}\right)^{\mathrm{T}} = \begin{bmatrix} a_1^1 \\ a_2^1 \\ a_3^1 \\ a_4^1 \\ a_5^1 \end{bmatrix} \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial z_1^2} & \frac{\partial \mathcal{L}}{\partial z_2^2} \end{bmatrix} = \begin{bmatrix} a_1^1 \cdot \frac{\partial \mathcal{L}}{\partial z_1^2} & a_1^1 \cdot \frac{\partial \mathcal{L}}{\partial z_2^2} \\ a_2^1 \cdot \frac{\partial \mathcal{L}}{\partial z_1^2} & a_2^1 \cdot \frac{\partial \mathcal{L}}{\partial z_2^2} \\ a_3^1 \cdot \frac{\partial \mathcal{L}}{\partial z_1^2} & a_3^1 \cdot \frac{\partial \mathcal{L}}{\partial z_2^2} \\ a_4^1 \cdot \frac{\partial \mathcal{L}}{\partial z_1^2} & a_4^1 \cdot \frac{\partial \mathcal{L}}{\partial z_2^2} \\ a_5^1 \cdot \frac{\partial \mathcal{L}}{\partial z_1^2} & a_5^1 \cdot \frac{\partial \mathcal{L}}{\partial z_2^2} \end{bmatrix}$$

Neural Networks: An Example with One Hidden Layer

- Similarly, the derivative of the loss with respect to $\mathbf{w}_0^2$ for one data point:

$$\frac{\partial \mathcal{L}_{(1)}}{\partial \mathbf{w}_0^2} = \frac{\partial \mathcal{L}_{(1)}}{\partial \mathbf{z}^2} = \begin{bmatrix} \frac{\partial \mathcal{L}_{(1)}}{\partial z_1^2} & \frac{\partial \mathcal{L}_{(1)}}{\partial z_2^2} \end{bmatrix}^{\mathrm{T}} = \begin{cases} \left[a_1^2 - 1, a_2^2\right]^{\mathrm{T}} & \text{when } y = 0 \\ \left[a_2^1, a_2^2 - 1\right]^{\mathrm{T}} & \text{when } y = 1 \end{cases}$$

- Then, the total loss of the data set,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_0^2} = \frac{1}{N} \sum_{i=1}^{N} \frac{\partial \mathcal{L}_{(i)}}{\partial \mathbf{w}_{0,(i)}^2} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial w_{0,1}^2} \\ \frac{\partial \mathcal{L}}{\partial w_{0,2}^2} \end{bmatrix}$$

Neural Networks: An Example with One Hidden Layer

- Now, for layer 1, the derivatives of the loss with respect to $\mathbf{a}^1$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^1} = \frac{\partial \mathbf{z}^2}{\partial \mathbf{a}^1} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{z}^2} = \mathbf{W}^2 \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{z}^2} = \begin{bmatrix} W_{11}^2 & W_{12}^2 \\ W_{21}^2 & W_{22}^2 \\ W_{31}^2 & W_{32}^2 \\ W_{41}^2 & W_{42}^2 \\ W_{51}^2 & W_{52}^2 \end{bmatrix} \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial z_1^2} \\ \frac{\partial \mathcal{L}}{\partial z_2^2} \end{bmatrix} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial a_1^1} \\ \frac{\partial \mathcal{L}}{\partial a_2^1} \\ \frac{\partial \mathcal{L}}{\partial a_3^1} \\ \frac{\partial \mathcal{L}}{\partial a_4^1} \\ \frac{\partial \mathcal{L}}{\partial a_5^1} \end{bmatrix}$$

Neural Networks: An Example with One Hidden Layer

- The derivatives of the loss with respect to $\mathbf{z}^1$ for one data point:

$$\frac{\partial \mathcal{L}_{(1)}}{\partial \mathbf{z}^1} = \frac{\partial \mathbf{a}^1}{\partial \mathbf{z}^1} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{a}^1} = \begin{bmatrix} \frac{\partial a_1^1}{\partial z_1^1} & 0 & \cdots & 0 \\ 0 & \frac{\partial a_2^1}{\partial z_2^1} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{\partial a_5^1}{\partial z_5^1} \end{bmatrix} \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial a_1^1} \\ \frac{\partial \mathcal{L}}{\partial a_2^1} \\ \frac{\partial \mathcal{L}}{\partial a_3^1} \\ \frac{\partial \mathcal{L}}{\partial a_4^1} \\ \frac{\partial \mathcal{L}}{\partial a_5^1} \end{bmatrix} = \begin{bmatrix} \frac{\partial a_1^1}{\partial z_1^1} \cdot \frac{\partial \mathcal{L}}{\partial a_1^1} \\ \frac{\partial a_2^1}{\partial z_2^1} \cdot \frac{\partial \mathcal{L}}{\partial a_2^1} \\ \frac{\partial a_3^1}{\partial z_3^1} \cdot \frac{\partial \mathcal{L}}{\partial a_3^1} \\ \frac{\partial a_4^1}{\partial z_4^1} \cdot \frac{\partial \mathcal{L}}{\partial a_4^1} \\ \frac{\partial a_5^1}{\partial z_5^1} \cdot \frac{\partial \mathcal{L}}{\partial a_5^1} \end{bmatrix}$$

Neural Networks: An Example with One Hidden Layer

- Finally, for layer 1, we have

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^1} = \mathbf{x} \left(\frac{\partial \mathcal{L}_{(1)}}{\partial \mathbf{z}^1} \right)^{\mathrm{T}} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \begin{bmatrix} \frac{\partial \mathcal{L}_{(1)}}{\partial z_1^1} & \frac{\partial \mathcal{L}_{(1)}}{\partial z_2^1} & \frac{\partial \mathcal{L}_{(1)}}{\partial z_3^1} & \frac{\partial \mathcal{L}_{(1)}}{\partial z_4^1} & \frac{\partial \mathcal{L}_{(1)}}{\partial z_5^1} \end{bmatrix}$$

- If we have N data points, then we can replace $\mathbf{x}$ with a $2 \times N$ matrix $\mathbf{X}$ and $(\partial \mathcal{L}_{(1)}/\partial \mathbf{z}^1)^{\mathrm{T}}$ would become an $N \times 5$ matrix.
- Thus, $\partial \mathcal{L}/\partial \mathbf{W}^1$ will still be a 2×5 matrix

Neural Networks: An Example with One Hidden Layer

- Similarly, the derivative of the loss with respect to $\mathbf{w}_0^1$ for one data point and for the entire data set,

$$\frac{\partial \mathcal{L}_{(1)}}{\partial \mathbf{w}_0^1} = \frac{\partial \mathcal{L}_{(1)}}{\partial \mathbf{z}^1} = \begin{bmatrix} \frac{\partial \mathcal{L}_{(1)}}{\partial z_1^1} \\ \frac{\partial \mathcal{L}_{(1)}}{\partial z_2^1} \\ \frac{\partial \mathcal{L}_{(1)}}{\partial z_3^1} \\ \frac{\partial \mathcal{L}_{(1)}}{\partial z_4^1} \\ \frac{\partial \mathcal{L}_{(1)}}{\partial z_5^1} \end{bmatrix}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{w}_0^1} = \sum_{i=1}^{N} \frac{\partial \mathcal{L}_{(i)}}{\partial \mathbf{w}_{0,(i)}^1} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial w_{0,1}^1} \\ \frac{\partial \mathcal{L}}{\partial w_{0,2}^1} \\ \frac{\partial \mathcal{L}}{\partial w_{0,3}^1} \\ \frac{\partial \mathcal{L}}{\partial w_{0,4}^1} \\ \frac{\partial \mathcal{L}}{\partial w_{0,5}^1} \end{bmatrix}$$

Neural Networks: An Example with One Hidden Layer

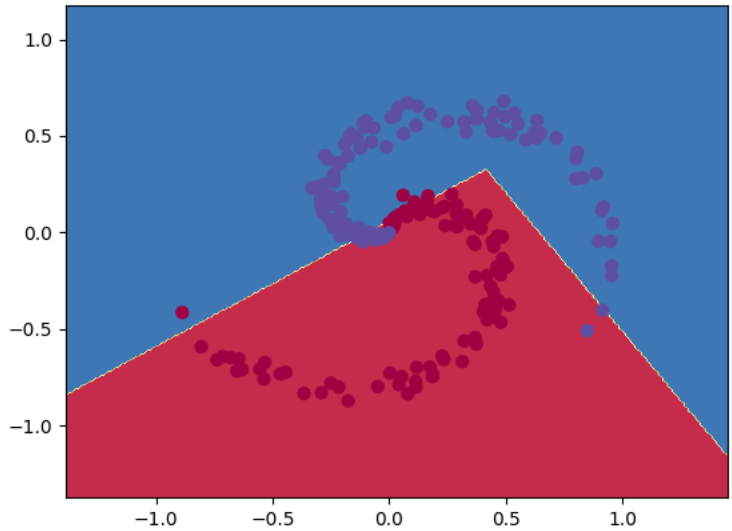
- Finally, we perform the stochastic gradient descent update for each layer

$$\mathbf{W}^1 = \mathbf{W}^1 - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^1}, \quad \mathbf{w}_0^1 = \mathbf{w}_0^1 - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}_0^1}$$
$$\mathbf{W}^2 = \mathbf{W}^2 - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^2}, \quad \mathbf{w}_0^2 = \mathbf{w}_0^2 - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}_0^2}$$

- Note that the dimensions of the matrices and vectors must match

Neural Networks: An Example with One Hidden Layer

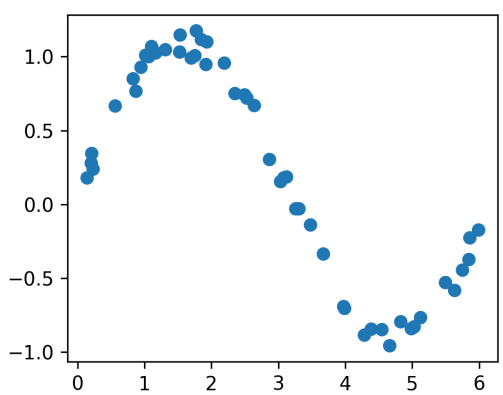
Result:

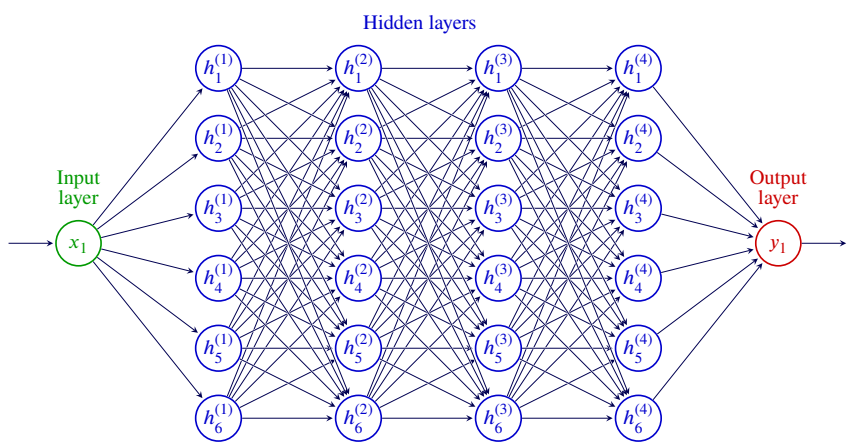


Download the Python script from BrightSpace

Neural Networks: Regression Example with PyTorch

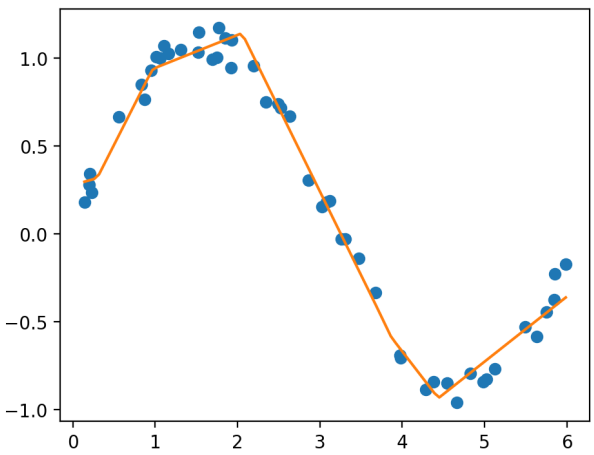
Goal: predict the function with a 4-layer neural network

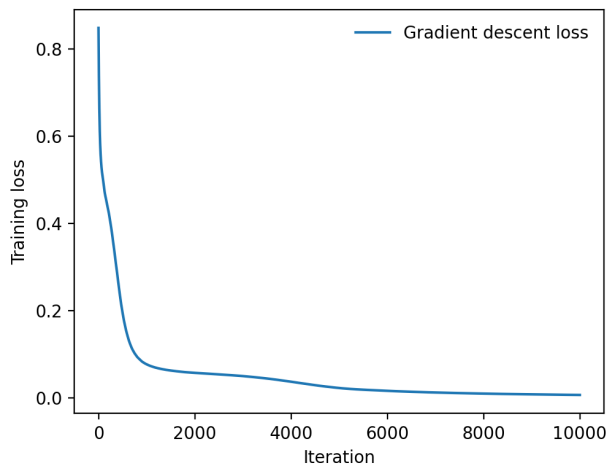




Neural Networks: Regression Example with PyTorch

Result:





Download the Python script from BrightSpace